

Algorithmique & Programmation

Introduction à la programmation pour le TAL

Eric Jordan

03 octobre 2025

- 1 Les variables
- 2 Contrôle de Flux
- 3 Passons à l'action

1 Les variables

2 Contrôle de Flux

3 Passons à l'action

Les types de données

Question : Quels types de données retrouve-t-on le plus souvent dans nos programmes ?

Les types de données

Question : Quels types de données retrouve-t-on le plus souvent dans nos programmes ?

- ▶ **string** : chaînes de caractères (lettres, chiffres, symboles)
- ▶ **int / float** : nombres entiers et réels
- ▶ **bool** : valeurs logiques (vrai / faux)

Les types de données

Question : Quels types de données retrouve-t-on le plus souvent dans nos programmes ?

- ▶ **string** : chaînes de caractères (lettres, chiffres, symboles)
- ▶ **int / float** : nombres entiers et réels
- ▶ **bool** : valeurs logiques (vrai / faux)

```
1  # string
2  nom = "Eric"
3
4  # int
5  nombre_de_langues = 4
6
7  # float
8  taille = 1.85
9
10 # bool
11 exp_en_python = True
```

Figure 1: Exemples dans un fichier Python

Les variables

Une fois qu'on sait les types de données qu'on aimerait traiter, ce serait bien de pouvoir les stocker : des variables !

Opérations de base :

- ▶ **Déclaration** : on réserve un espace en mémoire
- ▶ **Affectation (=)** : on associe une valeur à cette référence
- ▶ **Réaffectation/Mise à jour (=)** : on peut réassigner la variable vers une nouvelle valeur

'=' n'est pas pareil qu'aux maths !

Règles en Python :

- ▶ Doit commencer par une lettre ou un underscore (`_`)
- ▶ Peut contenir lettres, chiffres et underscores
- ▶ Sensible à la casse : `age` $\neq$ `Age`
- ▶ Ne doit pas être un mot réservé (`for`, `if`, `class`, ...)

Noms de variables

Règles en Python :

- ▶ Doit commencer par une lettre ou un underscore (`_`)
- ▶ Peut contenir lettres, chiffres et underscores
- ▶ Sensible à la casse : `age` $\neq$ `Age`
- ▶ Ne doit pas être un mot réservé (`for`, `if`, `class`, ...)

Styles d'écriture possibles :

- ▶ **CamelCase** : `monNomDeVariable`
- ▶ **PascalCase** : `MonNomDeVariable`
- ▶ **snake_case** : `mon_nom_de_variable`

Noms de variables

Règles en Python :

- ▶ Doit commencer par une lettre ou un underscore (`_`)
- ▶ Peut contenir lettres, chiffres et underscores
- ▶ Sensible à la casse : `age` $\neq$ `Age`
- ▶ Ne doit pas être un mot réservé (`for`, `if`, `class`, ...)

Styles d'écriture possibles :

- ▶ **CamelCase** : `monNomDeVariable`
- ▶ **PascalCase** : `MonNomDeVariable`
- ▶ **snake_case** : `mon_nom_de_variable` — **standard en Python**

Deux façons utiles d'affecter plusieurs variables en Python :

Deux façons utiles d'affecter plusieurs variables en Python :

1. Affectation simultanée

```
x = y = 5  
print(x)  # 5  
print(y)  # 5
```

Deux façons utiles d'affecter plusieurs variables en Python :

1. Affectation simultanée

```
x = y = 5  
print(x)  # 5  
print(y)  # 5
```

2. Affectation parallèle (déballage)

```
x, y = 5, 10  
print(x)  # 5  
print(y)  # 10
```

CG : Comment fonctionne une variable (en vrai) ?

Idée clé : Une variable n'est pas l'objet lui-même, mais une **référence en mémoire**.

CG : Comment fonctionne une variable (en vrai) ?

Idée clé : Une variable n'est pas l'objet lui-même, mais une **référence** en mémoire.



Figure 2: Une bibliothèque : l'adresse (rayon, étagère) pointe vers un livre précis

Traiter des floats et ints : Les opérateurs mathématiques

Opérateur	Fonction	Exemple	Résultat
+	Addition	$5 + 3$	8
-	Soustraction	$10 - 4$	6
*	Multiplication	$2 * 6$	12
/	Division	$15/3$	5
//	Division entière (quotient)	$7//2$	3
%	Modulo (reste de la division)	$7\%2$	1
**	Puissance	$2 * *3$	8

Table 1: Les opérateurs mathématiques

Traiter des floats et ints : Les opérateurs mathématiques

Opérateur	Fonction	Exemple	Résultat
+	Addition	$5 + 3$	8
-	Soustraction	$10 - 4$	6
*	Multiplication	$2 * 6$	12
/	Division	$15/3$	5
//	Division entière (quotient)	$7//2$	3
%	Modulo (reste de la division)	$7\%2$	1
**	Puissance	$2 * * 3$	8

Table 1: Les opérateurs mathématiques

Tableau de priorité des opérateurs en Python (sous 6.17 Operator precedence)

Traiter des floats et ints : Les opérateurs mathématiques

Opérateur	Fonction	Exemple	Résultat
+	Addition	$5 + 3$	8
-	Soustraction	$10 - 4$	6
*	Multiplication	$2 * 6$	12
/	Division	$15/3$	5
//	Division entière (quotient)	$7//2$	3
%	Modulo (reste de la division)	$7\%2$	1
**	Puissance	$2 * * 3$	8

Table 1: Les opérateurs mathématiques

Tableau de priorité des opérateurs en Python (sous 6.17 Operator precedence)

Le mieux c'est de mettre entre parenthèses

Fonctions natives de Python

Des choses pratiques dont on peut s'en servir partout en Python :

- ▶ `print(x)` - affiche la valeur de `x` dans le terminal
- ▶ `type(x)` - nous donne le type de `x`
- ▶ `range(x)` - nous donne toutes les ints de 0 à `x`
- ▶ `id(x)` - nous donne l'adresse en mémoire de `x`

Fonctions natives de Python

Des choses pratiques dont on peut s'en servir partout en Python :

- ▶ `print(x)` - affiche la valeur de `x` dans le terminal
- ▶ `type(x)` - nous donne le type de `x`
- ▶ `range(x)` - nous donne toutes les ints de 0 à `x`
- ▶ `id(x)` - nous donne l'adresse en mémoire de `x`

Vous allez rarement vous servir de `id()`, mais je vous le partage pour la démonstration

Exercice

En groupes de 3 :

- 1 print vos noms
- 2 print le nombre de langues que vous parlez
- 3 print la moyenne des langues parler entre vous

Sommaire

- 1 Les variables
- 2 Contrôle de Flux**
- 3 Passons à l'action

C'est très bien d'avoir un programme qui s'exécute de façon linéaire.

C'est très bien d'avoir un programme qui s'exécute de façon linéaire.
Mais on aimerait avoir un peu plus de contrôle.

C'est très bien d'avoir un programme qui s'exécute de façon linéaire. Mais on aimerait avoir un peu plus de contrôle.

- ▶ Le **chemin suivi par Python** s'appelle le **flux d'exécution**
- ▶ Les constructions qui modifient le programme sont appelées les **instructions de contrôle de flux**

Instruction If 1

- ▶ `if` (si) permet de tester la validité d'une condition, et exécuter du code spécifié **si la condition s'avère être vraie**

Instruction If 1

- ▶ if (si) permet de tester la validité d'une condition, et exécuter du code spécifié **si la condition s'avère être vraie**

```
if notre invité.e est veggie :  
    on ne lui sert pas de la viande !
```

Instruction If 1

- ▶ if (si) permet de tester la validité d'une condition, et exécuter du code spécifié **si la condition s'avère être vraie**

```
if notre invité.e est veggie :  
    on ne lui sert pas de la viande !
```

La Syntaxe :

```
a = 0  
if a > 0:  
    print("a est positif")
```

Instruction If 2

- ▶ `if` (si) permet de tester la validité d'une condition
- ▶ `elif` (sinon si) si la condition d'avant n'était pas vraie
- ▶ `else` (sinon) permet de programmer une exécution alternative

Instruction If 2

- ▶ `if` (si) permet de tester la validité d'une condition
- ▶ `elif` (sinon si) si la condition d'avant n'était pas vraie
- ▶ `else` (sinon) permet de programmer une exécution alternative

```
a = 0
if a > 0:
    print("a est positif")
elif a < 0:
    print("a est negatif")
else:
    print("a est nul")
```

Opérateurs de comparaison

Opérateur	Signification	Exemple	Résultat
==	Égal à	5 == 5	True
!=	Différent de	5 != 3	True
>	Supérieur à	7 > 2	True
<	Inférieur à	4 < 1	False
>=	Supérieur ou égal à	5 >= 5	True
<=	Inférieur ou égal à	3 <= 8	True

Table 2: Les opérateurs de comparaison

Instructions Imbriquées

On peut combiner des instructions, par exemple :

```
if embranchement == "vertebres":  
    if classe == "mammiferes":  
        if ordre == "carnivores":  
            if famille == "felins":  
                print("c'est peut-etre un chat")  
    elif classe == "oiseaux":  
        print("c'est peut-etre un canari")
```


Instructions Imbriquées

On peut combiner des instructions, par exemple :

```
if embranchement == "vertebres":  
    if classe == "mammiferes":  
        if ordre == "carnivores":  
            if famille == "felins":  
                print("c'est peut-etre un chat")  
        elif classe == "oiseaux":  
            print("c'est peut-etre un canari")
```

Ceci n'est pas du code valide Python

for

Permet de répéter une partie de notre code pour un nombre précis d'itérations

Quand on coupe du pain, on en coupe pour tout le monde :

pour chaque personne à la table : coupe une tranche de pain

for

Permet de répéter une partie de notre code pour un nombre précis d'itérations

Quand on coupe du pain, on en coupe pour tout le monde :

pour chaque personne à la table : coupe une tranche de pain

La syntaxe en Python :

```
for i in [0, 1, 2, 3]:  
    print("i a pour valeur ", i)
```

for

Permet de répéter une partie de notre code pour un nombre précis d'itérations

Quand on coupe du pain, on en coupe pour tout le monde :

pour chaque personne à la table : coupe une tranche de pain

La syntaxe en Python :

```
for i in [0, 1, 2, 3]:  
    print("i a pour valeur ", i)
```

Ou l'équivalent avec range() :

```
for i in range(4):  
    print("i a pour valeur ", i)
```

for

Permet de répéter une partie de notre code pour un nombre précis d'itérations

Quand on coupe du pain, on en coupe pour tout le monde :

pour chaque personne à la table : coupe une tranche de pain

La syntaxe en Python :

```
for i in [0, 1, 2, 3]:  
    print("i a pour valeur ", i)
```

Ou l'équivalent avec range() :

```
for i in range(4):  
    print("i a pour valeur ", i)
```

A tester :

```
for lettre in "hello":  
    print(lettre)
```

while (tant que)

Permet de répéter une partie du code tant que la condition qu'on spécifie est remplie

Tant qu'il reste de la pâte : versez en dans la poêle

while (tant que)

Permet de répéter une partie du code tant que la condition qu'on spécifie est remplie

Tant qu'il reste de la pâte : versez en dans la poêle

La syntaxe en Python :

```
i = 0
while i < 4:
    print("i a pour valeur ", i)
    i = i + 1
```

while (tant que)

Permet de répéter une partie du code tant que la condition qu'on spécifie est remplie

Tant qu'il reste de la pâte : versez en dans la poêle

La syntaxe en Python :

```
i = 0
while i < 4:
    print("i a pour valeur ", i)
    i = i + 1
```

Rassurez vous que votre boucle va terminer !

while (tant que)

Permet de répéter une partie du code tant que la condition qu'on spécifie est remplie

Tant qu'il reste de la pâte : versez en dans la poêle

La syntaxe en Python :

```
i = 0
while i < 4:
    print("i a pour valeur ", i)
    i = i + 1
```

Rassurez vous que votre boucle va terminer !

```
i = 0
while i < 4:
    print("i a pour valeur ", i)
```

Une suite dont chaque terme est la somme des 2 précédents :

```
a, b, c = 1, 1, 1
while b < 11:
    print(b)
    a, b, c = b, a+b, c+1
```

for vs while

En général **for** est utilisé quand on sait d'avance combien d'itérations on va faire, mais comme vous l'aurez vu les 2 sont interchangeables

Sommaire

- 1 Les variables
- 2 Contrôle de Flux
- 3 Passons à l'action**

Exercices

- 1 Faire un test pour savoir si le variable `a` est compris dans l'intervalle allant de 2 à 8
- 2 Afficher les 10 premiers termes de la table de multiplication par 5
- 3 Pareil que 2 mais ajouter un `*` pour tous les multiples de 3
- 4 Afficher une suite de 10 nombres dont chaque terme soit égale au triple du terme précédent

Merci, si vous avez des questions n'hésitez pas :

eric.jordan@inalco.fr